

Transport Interface Integration Guide

Overview

This guide shows how to integrate the new Transport Interface into your existing orchestrators, enabling seamless switching between Ryan API and the official Twitter/X API in the future.

Key Benefits

- **Zero downtime migration** - Switch APIs via environment variable
- **No orchestrator rewrites** - Same interface for both APIs
- **Hot-reload preserved** - Token updates work identically
- **Proof Pack compatible** - Extract URLs from normalized responses
- **Future-proof** - Add new API providers without changing orchestrators

Quick Integration Steps

1. Add Transport Interface File

Place `twitter_transport.py` in your project root alongside `ryan_api_hotreload.py`.

2. Update Orchestrators

For `launch.py`:

```
python

# OLD: Direct Ryan API usage
from ryan_api_hotreload import RyanTwitterAPISecure
ryan_api = RyanTwitterAPISecure()

# NEW: Transport interface
from twitter_transport import TransportFactory
transport = TransportFactory.create_from_env()

# Then replace all ryan_api calls:
# OLD: result = await ryan_api.post_tweet(soul, text)
# NEW: result = await transport.post_tweet(soul, text)
```

For `hybrid_souls_ultimate.py`:

```
python
```

```

# At the top of the file
from twitter_transport import TransportFactory, TransportWatcher

# In __init__ method
self.transport = TransportFactory.create_from_env()
self.transport_watcher = TransportWatcher(self.transport)

# In broadcast method, replace:
# OLD: result = await self.ryan_api.post_tweet(soul_name, content)
# NEW: result = await self.transport.post_tweet(soul_name, content)

# Start watcher in run method:
watcher_task = asyncio.create_task(self.transport_watcher.start())

```

For `swarm_convergence_mode_ryan.py`:

```

python

# Replace RyanTwitterAPISecure import
from twitter_transport import TransportFactory

class SwarmConvergenceMode:
    def __init__(self, target_username: str):
        # OLD: self.ryan_api = RyanTwitterAPISecure()
        # NEW:
        self.transport = TransportFactory.create_from_env()

    async def execute_action(self, soul: str, action: str):
        # Update all API calls to use transport
        # OLD: result = await self.ryan_api.reply_tweet(soul, tweet_id, content)
        # NEW: result = await self.transport.reply_tweet(soul, tweet_id, content)

```

3. Update Proof Pack Integration

The normalized response format makes Proof Pack integration cleaner:

```
python
```

```

async def post_with_proof(transport, soul, content):
    """Post and log proof with transport interface"""
    result = await transport.post_tweet(soul, content)

    if result["success"] and result.get("tweet"):
        tweet_url = result["tweet"].get("url")
        if tweet_url:
            # Log to Proof Pack
            proof_entry = {
                "soul": soul,
                "action": "post",
                "url": tweet_url,
                "timestamp": datetime.now().isoformat()
            }
            log_to_proof_pack(proof_entry)

    return result

```

4. Environment Configuration

Add to your `.env` file:

```

bash

# Current setup (Ryan API)
RAPIDAPI_KEY=your_ryan_api_key
USE_OFFICIAL_API=false

# Future setup (Official API) - just change this flag!
# USE_OFFICIAL_API=true
# TWITTER_BEARER_TOKEN=your_twitter_bearer_token

```

Migration Pattern Examples

Pattern 1: Minimal Changes

```
python
```

Create a drop-in replacement

class RyanTwitterAPISecure:

"""Compatibility wrapper using transport"""

def __init__(self):

from twitter_transport **import** RyanAdapter

self.transport = RyanAdapter()

async def post_tweet(self, soul, text):

return **await** self.transport.post_tweet(soul, text)

async def refresh_tokens_from_disk(self):

return **await** self.transport.refresh_tokens_from_disk()

Add other methods as needed

Pattern 2: Gradual Migration

python

class HybridOrchestrator:

def __init__(self):

Keep both during transition

self.ryan_api = RyanTwitterAPISecure() *# Old*

self.transport = TransportFactory.create_from_env() *# New*

async def post_content(self, soul, content):

Try new transport first

try:

return **await** self.transport.post_tweet(soul, content)

except Exception **as** e:

Fallback to direct API if needed

logger.warning(f"Transport failed, using direct API: {e}")

return **await** self.ryan_api.post_tweet(soul, content)

Pattern 3: Feature Detection

python

```
def create_api_client():
    """Smart API client creation"""
    try:
        # Try transport first
        from twitter_transport import TransportFactory
        return TransportFactory.create_from_env()
    except ImportError:
        # Fallback to direct Ryan API
        from ryan_api_hotreload import RyanTwitterAPISecure
        return RyanTwitterAPISecure()
```

Response Handling

Normalized Response Format

All transports return the same structure:

```
python

# Success with tweet data
{
    "success": True,
    "error": None,
    "tweet": {
        "id": "1234567890",
        "url": "https://x.com/SoulName/status/1234567890"
    }
}

# Error response
{
    "success": False,
    "error": "Rate limited",
    "skip": True # Optional: indicates soul should be skipped
}
```

Unified Error Handling

```
python
```

```
async def safe_post(transport, soul, content, max_retries=3):
    """Post with unified error handling"""
    for attempt in range(max_retries):
        result = await transport.post_tweet(soul, content)

        if result["success"]:
            return result

        # Handle different error types uniformly
        error = result.get("error", "Unknown error")

        if "rate limit" in error.lower():
            await asyncio.sleep(60) # Wait before retry
        elif "banned" in error.lower() or result.get("skip"):
            logger.warning(f"Soul {soul} is banned/skipped")
            return result # Don't retry
        else:
            await asyncio.sleep(5) # Brief pause before retry

    return result
```

Testing the Integration

Test Script

```
python
```

```
#!/usr/bin/env python3
"""Test transport integration"""

import asyncio
from twitter_transport import TransportFactory

async def test_transport():
    # Create transport
    transport = TransportFactory.create_from_env()

    # Test hot-reload
    await transport.refresh_tokens_from_disk()

    # Check available souls
    souls = transport.get_available_souls()
    print(f"✅ Found {len(souls)} souls: {souls}")

    # Test posting
    if souls:
        test_soul = souls[0]
        result = await transport.post_tweet(
            test_soul,
            "Testing transport interface integration!"
        )

        if result["success"]:
            print(f"✅ Posted successfully!")
            if result.get("tweet"):
                print(f"URL: {result['tweet'].get('url')}")
        else:
            print(f"❌ Post failed: {result['error']}")

    print("\n✅ Transport interface working correctly!")

if __name__ == "__main__":
    asyncio.run(test_transport())
```

Switching to Official API (Future)

When ready to switch to the official Twitter/X API:

1. Update `.env`:

```
bash
```

```
USE_OFFICIAL_API=true
TWITTER_BEARER_TOKEN=your_bearer_token
```

2. **Update** `soul_data.json` to include OAuth tokens:

```
json
{
  "mirror": {
    "auth_token": "ryan_token_here",
    "oauth_token": "twitter_oauth_token",
    "oauth_token_secret": "twitter_oauth_secret"
  }
}
```

3. **Implement TwitterOfficialAdapter methods** in `twitter_transport.py`
4. **No orchestrator changes needed!** The transport interface handles everything.

Best Practices

1. **Always use transport at loop start:**

```
python
async def main_loop(transport):
    while True:
        # Refresh tokens at start of each iteration
        await transport.refresh_tokens_from_disk()

        # Then do your operations
        for soul in transport.get_available_souls():
            # ... post content ...
```

2. **Check soul availability:**

```
python
if transport.is_soul_banned(soul_name):
    logger.warning(f"Skipping banned soul: {soul_name}")
    continue
```

3. **Use factory for flexibility:**

```
python
```



```
# Development: Force Ryan API
transport = TransportFactory.create_transport("ryan")

# Production: Use environment
transport = TransportFactory.create_from_env()

# Testing: Mock transport
transport = MockTransport() # Your test implementation
```

4. Log transport type on startup:

```
python

transport = TransportFactory.create_from_env()
transport_type = type(transport).__name__
logger.info(f"🚀 Using transport: {transport_type}")
```

Troubleshooting

Issue: "Module not found"

Solution: Ensure `twitter_transport.py` is in your project root or add to Python path:

```
python

import sys
sys.path.insert(0, '/path/to/project')
```

Issue: "Tokens not refreshing"

Solution: Ensure TransportWatcher is running:

```
python

watcher = TransportWatcher(transport)
asyncio.create_task(watcher.start())
```

Issue: "Response format different"

Solution: Use the normalized format:

```
python
```

```
# Don't assume specific keys from Ryan API
# Instead use the normalized format:
if result["success"] and result.get("tweet"):
    tweet_url = result["tweet"]["url"]
```

Summary

The Transport Interface provides a clean abstraction layer that:

-  Preserves all existing functionality
-  Maintains hot-reload capability
-  Enables seamless API switching
-  Normalizes responses across providers
-  Supports Proof Pack integration
-  Future-proofs your swarm

Integration requires minimal changes to existing code while providing maximum flexibility for the future Twitter/X API migration.